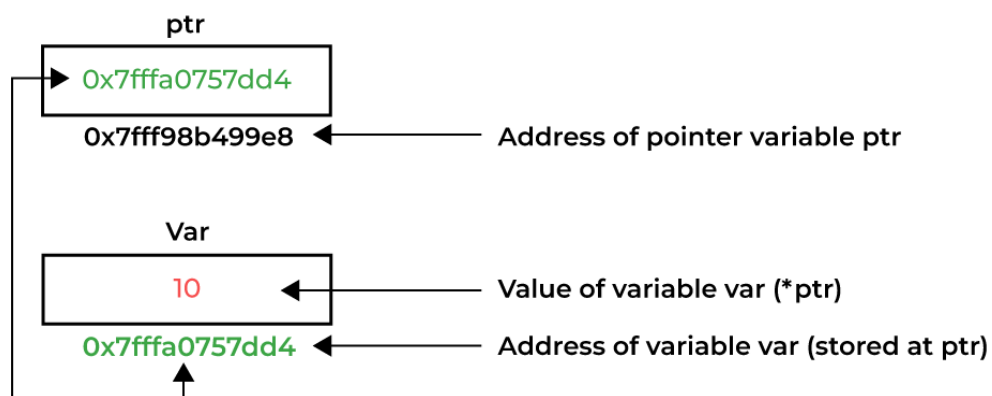


➤ Pointer in C :-

What is a Pointer :-

- A pointer stores the memory address of another variable.
- Declared using * symbol (e.g., int *ptr;)
- A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location
- The pointer in C language is a variable which stores the address of another variable.
- This variable can be of type int, char, array, function, or any other pointer.



Advantages of Pointer :-

- Pointer reduces the code and improves the performance.
- It makes you able to access any memory location.
- We can return multiple values from a function.

1.Declaration of Pointers:-

A pointer must specify which data type it will point to.

Syntax:

datatype *pointer_name;

- datatype → the type of data the pointer will store the address for.
- * → indicates it's a pointer.

Example :-

```
int *ptr;           // Pointer to integer
float *fptr;        // Pointer to float
char *cptr;         // Pointer to character
double *dptr;       // Pointer to double
int **pp;           // Pointer to Pointer
```

2. Assigning Address :-

```
int x=10;  
int *p=&x; // p stores the address of x
```

3. Format Specifiers for Pointers :-

In print, we use specific format specifiers:

Specifier	Used For
%p	To print an address (pointer value)
%d	To print the value (if pointer is dereferenced and data is int)
%c	For char value (when dereferencing char *)
%f	For float/double (when dereferencing)
%u	To print an unsigned int (non-negative integer) in decimal format.

4. * and & Operators :-

- & → Address-of operator: Gives the memory address of a variable.
- → Dereference operator: Accesses the value stored at a memory address.

Example ;

```
#include <stdio.h>  
int main() {  
    int x = 5;  
    int *ptr = &x;  
    printf("Address of x: %p\n", &x);  
    printf("Value of ptr (address): %p\n", ptr);  
    printf("Value of x using *ptr: %d\n", *ptr);  
    return 0;  
}
```

5. Null Pointer :-

- A null pointer points to nothing (address 0).
- Used to indicate a pointer is not assigned yet.
- Accessing *ptr when it's NULL will cause a runtime error (segmentation fault).

Example :-

```
#include <stdio.h>  
int main()  
{  
    int *ptr = NULL;  
    if (ptr == NULL)  
    {  
        printf("Pointer is NULL\n");  
    }  
}
```

```

}
return 0;
}

```

6.sizeof() Operator with Pointers:

- sizeof() tells the number of bytes a variable or data type occupies.
- On a 32-bit system, it usually prints 4 bytes.
- On a 64-bit system, it usually prints 8 bytes, regardless of the data type.

Example :-

```

#include <stdio.h>
int main() {
    int *p;
    double *q;
    printf("Size of int pointer: %u bytes\n", sizeof(p));
    printf("Size of double pointer: %u bytes\n", sizeof(q));
    return 0;
}

```

7.Pointer Arithmetic :-

- You can add/subtract integers to/from pointers.
- When you do this, it moves by the size of the data type.

Example :-

```

#include <stdio.h>
int main()
{
    int arr[3] = {10, 20, 30};
    int *ptr = arr;
    printf("%d\n", *ptr);
    printf("%d\n", *(ptr + 1));
    printf("%d\n", *(ptr + 2));
    ptr++;
    printf("After ptr++: %d\n", *ptr);
    return 0;
}

```

8.Basic pointer :-

Example :-

```

#include <stdio.h>
int main()
{
    int a = 10;
}

```

```

int *ptr;
ptr = &a;
printf("Value of a: %d\n", a);
printf("Address of a: %p\n", &a);
printf("Pointer value (address): %p\n", ptr);
printf("Value via pointer: %d\n", *ptr);
*ptr = 20;
printf("New value of a: %d\n", a);
return 0;
}

```

9.Pointer with Function (Pass by Reference) :-

Example :-

```

#include <stdio.h>
void changeValue(int *x)
{
    *x = 100;
}
int main()
{
    int num = 50;
    printf("Before: %d\n", num);
    changeValue(&num);
    printf("After: %d\n", num);
    return 0;
}

```

10.Accessing Array using Pointer:

Example :-

```

#include <stdio.h>
int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;
    printf("Access elements using pointer:\n");
    for (int i = 0; i < 5; i++)
    {
        printf("arr[%d] = %d (Address: %p)\n", i, *(ptr + i), (ptr + i));
    }
    return 0;
}

```